Innovation

Mobile Applications Development

Student: G00474271

1 Introduction

This document describes the extra features I added to my project, on top of what the brief asked for. The features are grouped into three sections, namely user experience, visual style, and code structure.

2 User experience improvements

1. Loading spinner during API calls

When the app fetches movies from the API, there might be a small wait while the data comes back. To show the user that something is happening, I added a spinner that appears during this wait in the home page.

I have done this by setting a boolean called loading to true before the API call, and back to false once the data arrives. The template uses *ngIf to show the spinner only when loading is true.

```
async getTrending() {
  this.loading = true; //start spinner
  //reset the heading whenever trending is loaded
  this.heading = "Today's Trending Movies";
  let options: HttpOptions = {
    url: 'https://api.themoviedb.org/3/trending/movie/day?api_key=' + this.API_KEY
  };
  let result = await this.mhs.get(options);
  //result.data holds the JSON body, the array we need is inside .results
  this.movies = result.data.results;
  //verifies the movies array is populated correctly
  this.loading = false;//stop spinner
  console.log(this.movies);
```

```
<div *ngIf="loading">
  <ion-spinner name="crescent"></ion-spinner>
  <p>Loading...</p>
</div>
```

On a fast internet connection, the spinner only shows for a fraction of a second and is hard to see. To check if it works, I used the network throttling tool in Edge developer tools. By opening DevTools, going to the Network tab, and changing the throttling dropdown from “No throttling” to “Slow 4G”, the API call slows down to a few seconds. The spinner then becomes clearly visible while the data loads, and disappears once the cards appear.

References:

https://ionicframework.com/docs/api/spinner

2. Empty state messages

There are some places in the app where data might not be available, and instead of showing a blank screen, I show a message.

On the Favourites page, when nothing has been saved, the user sees “No favourites yet”. This uses *ngIf to check if the favourites array is empty:

```
<div *ngIf="favourites.length === 0">
  <h2>No favourites yet</h2>
</div>
```

On the home page, when a search returns nothing, the message “No movies found for [keyword]” appears with the search word filled in. This uses two conditions joined with &&: the movies array must be empty AND there must be a search string. The second check makes sure the message does not appear on first page load before we have searched.

```
<div *ngIf="movies.length === 0 && searchString">
  <h3>No movies found for "{{ searchString }}"</h3>
</div>
```

3. Quick remove button on Favourites page

On the Movie Details page, there is a button to add or remove a movie from favourites. But to remove a movie that way, we have to first click the movie, then click the remove button, and then go back.

To make this faster, I added a small red bin icon on each card on the Favourites page. Clicking it removes that one movie straight away, without needing to open the movie details.

In the template, the button is placed inside each card next to the Details button:

```
<ion-button (click)="openMovie(movie)">Details</ion-button>
<ion-button fill="clear" color="danger" (click)="removeFromFavourites(movie)">
  <ion-icon name="trash" slot="icon-only"></ion-icon> <!--https://ionicframework.com/docs/api/buttons-->
</ion-button>
```

The attributes do the following:

- **`fill="clear"`** removes the button’s background, so only the icon is shown.
- **`color="danger"`** makes the icon red.
- **`slot="icon-only"`** tells Ionic the button contains only an icon, so the spacing is set correctly.
- **`(click)="removeFromFavourites(movie)"`** calls the method below, passing in the specific movie that was clicked.

The method in the TypeScript file removes the movie from storage and refreshes the list:

```
async removeFromFavourites(movie: any) {
  await this.mds.removeFavourite(movie.id);
  this.favourites = await this.mds.getFavourites();
}
```

The first line calls the helper method on `MyDataService` to delete the movie from the favourites array in storage. The second line reloads the updated array so the UI updates immediately and the card disappears.

References:

https://ionicframework.com/docs/api/buttons
https://ionicframework.com/docs/api/icon
https://ionic.io/ionicons

4. Smart heading and search reset on home page

The heading on the home page is not a fixed string. It changes based on what the user is doing. If they are browsing trending movies, it reads “Today's Trending Movies”. If they have searched for something, it reads “[searched word] Movies”. If the search does not return anything it returns the fall back “No movies found for [searched word]”

I made this work using a property called heading: **heading: string = "Today's Trending Movies"** in the ts file.

The heading is updated only inside the methods that actually run a search or fetch trending movies, not whenever the user types in the search box:

```
async getTrending() {
  this.loading = true; //start spinner
  //reset the heading whenever trending is loaded
  this.heading = "Today's Trending Movies";
```

```
async search() {
  //fall back to trending if the search box is empty or whitespace
  if (this.searchString.trim() === '') {
    this.getTrending();
    return;
  }
  this.loading = true; //start spinner
  //update the heading to reflect the actual search performed
  this.heading = this.searchString + ' Movies';
```

This avoids a small bug I had in the heading. Now the heading only changes when the cards on screen actually change.

```
<h2 *ngIf="movies.length > 0">{{ heading }}</h2> <!-- amended instance-->
```

The ***ngIf="movies.length > 0"** hides the heading when there are no results to show. In that case, the “No movies found for [keyword]” (see previous explanation) message takes over.

I also added one line in ionViewWillEnter that resets the search box when the user comes back to the home page, so the old search text does not stay in the box:

```
ionViewWillEnter() {
  this.getTrending();
  this.searchString = '';
}
```

3 Visual style

5. Different button styles for add and remove

The ADD TO FAVOURITES and REMOVE FROM FAVOURITES buttons look different on purpose, so the user can see at a glance what action they are about to take.

The ADD button is solid blue with a filled heart icon. The REMOVE button has a white background with a blue border, and uses a broken-heart icon.

In the template, the two buttons are placed next to each other, but only one is visible at a time depending on whether the movie is already a favourite:

```
<ion-button (click)="toggleFavourite()" *ngIf="!isFav">
  <ion-icon name="heart" slot="start"></ion-icon>
  Add To Favourites
</ion-button>
<ion-button fill="outline" (click)="toggleFavourite()" *ngIf="isFav"> <!-- https://ionicframework.com/docs/api/input for fill="outline" -->
  <ion-icon name="heart-dislike" slot="start"></ion-icon> <!-- https://ionic.io/ionicons -->
  Remove From Favourites
</ion-button>
```

The differences between the two buttons are:

- The first button has no fill attribute, so it uses the default solid blue style.
- The second button has **fill="outline"**, which gives it a white background with a blue border instead of a solid fill.
- The icon names are different: heart for the filled heart, and heart-dislike for the broken-heart variant.

Both buttons call the same toggleFavourite() method when clicked. The method checks the current state and calls either addFavourite or removeFavourite on MyDataService:

```
async toggleFavourite() {
  if (this.isFav) {
    await this.mds.removeFavourite(this.movie.id);
    this.isFav = false;
  } else {
    await this.mds.addFavourite(this.movie);
    this.isFav = true;
  }
}
```

The isFav boolean is set when the page loads, by checking storage:

```
async loadMovie() {
  this.movie = await this.mds.get('selectedMovie');
  console.log(this.movie); //checking if movie obj's been captured
  await this.loadCredits();
  this.isFav = await this.mds.isFavourite(this.movie.id); //calling helper
}
```

References:

https://ionicframework.com/docs/api/button
https://ionic.io/ionicons
https://ionicframework.com/docs/api/input

6. Date formatting

Birthdays and dates of death from the API come in a format like `1956-07-09`, which is hard to read. To make these easier to understand, I used Angular's DatePipe with the format `dd/MM/yyyy`. This matches the day-month-year style used in Ireland.

In the template, I applied the pipe |, followed by the pipe name and the format string:

```
<p *ngIf="person.birthday">
  {{ person.birthday | date: 'dd/MM/yyyy' }}<span *ngIf="person.deathday"> - {{ person.deathday| date: 'dd/MM/yyyy' }}</span>
</p>
```

The pipe is applied directly in the template using {{ date | date: 'dd/MM/yyyy' }}, and the dates are wrapped in ***ngIf** so they only show when they exist.

References:

https://forum.ionicframework.com/t/formatting-date-time-for-display/60671/5
https://docs.angularjs.org/api/ng/filter/date

https://stackoverflow.com/questions/47035327/ionic-date-pipe-and-local-date

https://angular.io/api/common/DatePipe

7. Avatar for cast and crew

For the cast and crew lists on the Movie Details page, and the Other Movies list on the Person Details page, I used **ion-avatar** to display each person's photo or each movie's poster as a circle. This was not in the brief, but it makes the lists much nicer to look at (in my humble opinion).

Below is how I built the cast list. Each cast member is one ion-item, and the avatar is placed inside it with **slot="start"**, which positions it on the left:

```
<h2>Cast</h2>
<ion-list>
  <ion-item *ngFor="let person of cast" (click)="selectPerson(person)">
    <ion-avatar slot="start">
      <!-- show image if profile_path exists, otherwise show "No image" -->
      <img *ngIf="person.profile_path" src="https://image.tmdb.org/t/p/w185{{ person.profile_path }}" />
      <p *ngIf="!person.profile_path">No image</p>
    </ion-avatar>
    <ion-label>
      <h3>{{ person.character }}</h3>
      <p>{{ person.name }}</p>
    </ion-label>
  </ion-item>
</ion-list>
```

References:

https://ionicframework.com/docs/api/avatar

8. Use of different TMDB image sizes

The TMDB API returns image paths in the form `/abc123.jpg`. To get a working image URL, we have to add a base URL and a size code in front of the path, like this:
`https://image.tmdb.org/t/p/[size]/[poster_path]`

The size codes are predefined values such as `w185`, `w342`, `w500`, `w780`, and `original`. Other values may give a 404 error or be removed at any time.

In my app, I used `w500` for full-size posters on the home page, Movie Details page, and Favourites page:

I used `w185` for the small avatar thumbnails in cast and crew lists, since the smaller image loads faster and uses less data without losing quality at thumbnail size.

I previously used w200 because I assumed any number after the "w" would work, since it just refers to the image width in pixels. I did a quick search and also showed it being used in some online code examples, so I went with it. However, after reading the TMDB documentation and configuration endpoint more carefully, I found that w200 is not in the official supported list. Hence, I thought that unsupported sizes can stop working at any time, so I changed all w200 URLs to w185, which is the closest officially supported size and looks visually identical at thumbnail size.

References:

https://developer.themoviedb.org/docs/image-basics
https://developer.themoviedb.org/reference/configuration-details

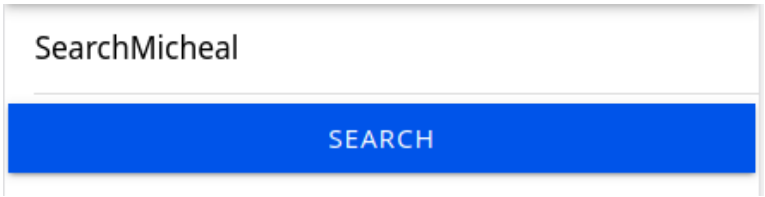
9. Search bar layout fix

When an `ion-label` and an `ion-input` are placed in the same `ion-item`, they end up overlapping. The label and the input fight for space and the result looks messy.

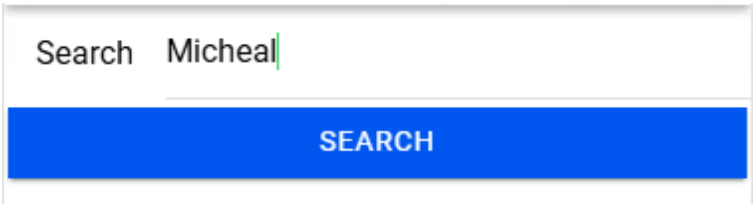
To fix this, I added a small inline style on the label using flexbox:

```
<ion-item>
  <ion-label slot="start" style="flex: 0 0 auto; margin-right: 16px;">Search</ion-label>
  <ion-input [(ngModel)]="searchString" placeholder="Movie Name"></ion-input>
</ion-item>
```

BEFORE



AFTER



The styles do two things:

- **`flex: 0 0 auto`** is shorthand for three flex properties. The first `0` means the label should not grow to fill extra space. The second `0` means it should not shrink either. The `auto` means it should size itself to fit its own content (the word “Search”). Together these stop the label from stretching across the row.
- **`margin-right: 16px`** adds a 16-pixel gap to the right of the label, separating it from the input.

References:

https://ionicframework.com/docs/api/label
https://developer.mozilla.org/en-US/docs/Web/CSS/flex

4 Code structure

10. Helper methods in MyDataService

The MyDataService class is structured in two layers.

The first layer is made up of two generic methods, **set(key, value)** and **get(key)**, which wrap Ionic Storage and can store any value under any key.

On top of these, I added four helper methods just for handling favourites: **getFavourites**, **addFavourite**, **removeFavourite**, and **isFavourite**. The helpers internally call set and get, but add the extra logic needed for managing the favourites list, such as reading the current array, modifying it, and saving it back.

This means each page only needs to call one method, instead of repeating the same code in different places. If I ever need to change how favourites are stored, I only have to change it in one place.

Below is the removeFavourite method as an example. It reads the current list of favourites, builds a new list that excludes the one we want to remove, and saves the result back to storage:

```
async removeFavourite(movieId: number) {
  //get the current list of favourites
  let favs = await this.getFavourites();
  //create a new empty array to hold the movies we want to keep
  let newFavs = [];
  //loop through every movie currently saved
  for (let i = 0; i < favs.length; i++) {
    //if the current movie is NOT the one we want to remove, keep it
    if (favs[i].id !== movieId) {
      newFavs.push(favs[i]);
    }
  }
  //save the new array (without the removed movie) back to storage
  await this.set('favourites', newFavs);
}
```

The helper is written using a simple for loop rather than more advanced array methods like filter. The loop checks each saved movie one by one. If the current movie's id is not the one we want to remove, it is added to a new array. Once the loop finishes, the new array (without the removed movie) is saved back to storage under the key **'favourites'**.

The isFavourite method follows the same pattern. It loops through the list and returns true as soon as it finds a match, or false if the loop finishes without one:

```
async isFavourite(movieId: number): Promise<boolean> {
  //get the current list of favourites
  let favs = await this.getFavourites();
  //loop through every saved movie
  for (let i = 0; i < favs.length; i++) {
    //if the current movie's id matches the one we're checking, it's a favourite
    if (favs[i].id === movieId) {
      return true;
    }
  }
  //if the loop finishes without finding a match, the movie is not a favourite
  return false;
}
```

Pages then use these helpers from inside their own methods. For example, on the Movie Details page, the favourite status is loaded once when the page opens (using isFavourite), and the toggle button calls addFavourite or removeFavourite depending on the current state.